Individual Contribution Report

Student Name: Matthew Sto. Tomas
Group Name: MiniOS Alpha
Assigned Role: CLI Developer

1. What did I contribute to the project?

• Designed and developed the central command-line interface (CLI) and interactive Read-Eval-Print Loop (REPL) in `main.py`.
• Built the command tokenizer, argument parser, and extensible dictionary-based command dispatch router.
• Implemented the core general commands: `help`, `clear` / `cls`, `exit` / `shutdown`, `whoami`, `systeminfo`, `date`, `time`, `history`, `echo`, and `version`.
• Established the defensive error handling barrier and static parameter guards (`need`, `to_int`) ensuring the shell never crashes on invalid inputs or missing arguments.
• Integrated and coordinated the submodules developed by teammates (`filesystem.py`, `processes.py`, `memory.py`, `users.py`) into the main shell execution loop.

2. What features did I personally develop?

• **`MiniOS.shell_loop()`**: Interactive command prompt loop displaying `MiniOS> `, capturing user input, stripping whitespace, ignoring empty inputs, and tracking session command history.
• **`MiniOS.execute(line)`**: Command tokenizer and dispatch engine that normalizes command names to lowercase, routes calls via the command dictionary (`self.commands`), and wraps execution in a global exception barrier.
• **`MiniOS.__init__()`**: Simulator initialization registering all command handlers and mapping natural command aliases (`cls` -> `clear`, `shutdown` -> `exit`, `ls` -> `dir`, `login` -> `logout`).
• **`MiniOS.need()` & `MiniOS.to_int()`**: Static validation guards verifying required argument counts with syntax hints and safely parsing numeric inputs without crashing.
• **`cmd_help()`**: Displays a formatted two-column command catalog grouped into General, Files, Processes, Memory, and Users.
• **`cmd_clear()`**: Dynamically clears the console buffer using `os.name` (`cls` on Windows CMD, `clear` on Unix).
• **`cmd_exit()`**: Controlled simulator shutdown sequence printing an exit confirmation and stopping the execution loop cleanly.
• **`cmd_systeminfo()`**: Real-time dashboard synthesizing metrics across all subsystems (OS version, architecture, CPU, RAM pool, running processes, filesystem counts, and uptime).
• **`cmd_whoami()`, `cmd_date()`, `cmd_time()`, `cmd_history()`**: Session utilities displaying the current user, formatted date/time, and a chronological history of entered commands.

3. What problems did I encounter?

• **Cross-Platform Screen Clearing**: console clear commands caused errors across operating systems. `cls` failed in Unix during development, while `clear` was unrecognized in Windows CMD.

• **Cluttered code**: Using nested `if/elif/else` statements for 20+ commands created cluttered code, slowed command dispatch, and generated git merge conflicts during team integration.

4. How did I solve those problems?

• **Dynamic OS Detection & Aliasing**: Used Python's standard `os.name` to invoke `cls` on Windows and `clear` on Unix, and registered both as first-class aliases in `self.commands`.
• **Reusable Guards & Exception Barrier**: Implemented `need()` and `to_int()` to validate inputs before execution, and wrapped command execution in a global `try...except` barrier in `execute()` to catch subsystem exceptions and generic errors gracefully.
• **Signal Trapping in Shell Loop**: Encapsulated the main session loop in `start()` inside a `try...except (EOFError, KeyboardInterrupt)` block, printing a clean shutdown sequence and exiting gracefully.
• **Dictionary-Based Dispatch Table**: Refactored command routing into an O(1) hash map (`self.commands`), cleanly decoupling command registration from execution and simplifying teammate module integration.

5. What did I learn from this project?

• **Shell Architecture & REPL Pattern**: Gained practical knowledge of how command-line interpreters tokenize input, manage execution state, and act as the gateway to operating system services.
• **Fault Tolerance & Defensive Programming**: Learned why operating system shells must be strictly fault-tolerant—user errors and malformed commands must never crash the operating system; errors must be intercepted, diagnosed, and recovered from cleanly.
• **Modular System Integration**: Strengthened skills in collaborative architecture by establishing clean interface contracts and standardized exception conventions across independent modules.
• **Cross-Platform Portability**: Developed practical skills in handling platform-specific differences (terminal commands, path handling) to ensure consistent execution in Windows Command Prompt.

6. Evidence of my contribution

1: Help command output (`help`)



2: Defensive error handling (invalid inputs)

